

# CLI reference

## calepin

Preprocess Typst documents with executable code chunks

Usage: calepin <COMMAND>

Commands:

|         |                                                                    |
|---------|--------------------------------------------------------------------|
| new     | Create a new example Typst file or website scaffold                |
| health  | Check Calepin's local runtime environment and local links          |
| compile | Preprocess, then invoke typst compile                              |
| watch   | Watch, preprocess, and delegate recompiles to typst watch          |
| serve   | Serve static files locally                                         |
| stop    | Stop a running calepin watch process                               |
| clean   | Remove <code>`.calepin`</code> directories and generated artifacts |
| help    | Print this message or the help of the given subcommand(s)          |

Options:

|               |               |
|---------------|---------------|
| -v, --version | Print version |
| -h, --help    | Print help    |

## calepin new

Create a new example Typst file or website scaffold

Usage: calepin new [OPTIONS] <PATH> [OUTPUT]

Arguments:

|          |                                                                                                                                 |
|----------|---------------------------------------------------------------------------------------------------------------------------------|
| <PATH>   | Path to the new .typ file, or <code>`website`</code> / <code>`academic`</code> / <code>`theme`</code> to scaffold project files |
| [OUTPUT] | Destination directory when PATH is <code>`website`</code> or <code>`academic`</code>                                            |

Options:

|                 |                                                                            |
|-----------------|----------------------------------------------------------------------------|
| -f, --force     | Overwrite the file if it already exists                                    |
| --theme <THEME> | Builtin theme to copy when PATH is <code>`theme`</code> (default: calepin) |
| -h, --help      | Print help                                                                 |

## calepin health

Check Calepin's local runtime environment and local links

Usage: calepin health [OPTIONS]

Options:

|                        |                                              |
|------------------------|----------------------------------------------|
| --config <CONFIG>      | Path to project config TOML                  |
| -d, --depth <DEPTH>    | Maximum recursion depth when searching links |
| --json                 | Print machine-readable JSON                  |
| --strict               | Exit with an error when warnings are present |
| --check-external-links | Also check external links                    |
| -h, --help             | Print help                                   |

## calepin compile

Preprocess, then invoke typst compile

Usage: calepin compile [OPTIONS] <INPUT> [OUTPUT] [TYPST\_ARGS]...

#### Arguments:

<INPUT>

Input .typ file, or a website source directory containing calepin.toml

[OUTPUT]

Output file path, or website output directory when INPUT is a directory

[TYPST\_ARGS]...

Arguments forwarded to typst compile after `--`

#### Options:

--format <FORMAT>

Output format passed to typst compile

[possible values: pdf, png, svg, html]

--theme <THEME>

Theme bundle: a builtin name (calepin, academic), a path to a theme directory, or false

--minify

Minify HTML output after theming and asset processing

--config <CONFIG>

Path to project config TOML

-q, --quiet

Quiet mode

--timeout <TIMEOUT>

Per-chunk timeout in seconds

-P, --param <KEY=VALUE>

Override a document parameter as `key=value` (repeatable).

Takes precedence over `calepin.setup(params: ...)` , so the same document can render with different values without editing the source.

-h, --help

Print help (see a summary with '-h')

## calepin watch

Watch, preprocess, and delegate recompiles to typst watch

Usage: calepin watch [OPTIONS] <INPUT> [OUTPUT] [TYPST\_ARGS]...

#### Arguments:

<INPUT>

Input .typ file, or a website source directory containing calepin.toml

[OUTPUT]

Output file path, or website output directory when INPUT is a directory

[TYPST\_ARGS]...

Arguments forwarded to typst watch after `--`

#### Options:

- `--format <FORMAT>`  
Output format passed to typst watch  
  
[possible values: pdf, png, svg, html]
- `--serve`  
Serve the website while watching a directory
- `--open`  
Open the served website in the default browser
- `--host <HOST>`  
Interface to bind when serving a watched website  
  
[default: 127.0.0.1]
- `--port <PORT>`  
Port to bind when serving a watched website (default: first free port from 8000)
- `--config <CONFIG>`  
Path to project config TOML
- `-q, --quiet`  
Quiet mode
- `--timeout <TIMEOUT>`  
Per-chunk timeout in seconds
- `-P, --param <KEY=VALUE>`  
Override a document parameter as `key=value` (repeatable).  
  
Takes precedence over `calepin.setup(params: ...)` , so the same document can render with different values without editing the source.
- `-h, --help`  
Print help (see a summary with '-h')

## calepin serve

Serve static files locally

Usage: calepin serve [OPTIONS] <DIR>

#### Arguments:

<DIR> Directory containing static files to serve

#### Options:

- `--host <HOST>` Interface to bind [default: 127.0.0.1]
- `-p, --port <PORT>` Port to bind (default: first free port from 8000)
- `--open` Open the website in the default browser
- `-h, --help` Print help

## **calepin stop**

Stop a running calepin watch process

Usage: calepin stop [INPUT]

Arguments:

[INPUT] Input .typ file to stop the matching calepin watch. Omit this value to stop all active watches under the current project's `.calepin` directory

Options:

-h, --help Print help

## **calepin clean**

Remove `.calepin` directories and generated artifacts

Usage: calepin clean [OPTIONS]

Options:

-d, --depth <DEPTH> Maximum recursion depth when searching for `.calepin` directories

-y, --yes Skip interactive confirmation and delete immediately

-h, --help Print help